TPRE521

Gordillo Loïs, Boufenouche Mehdi, Momo Carnegie & Moragny Alban

Spécification technique

CRÉER UN MODÈLE DE DONNÉES D'UNE SOLUTION I.A
EN UTILISANT DES MÉTHODES DE DATA SCIENCE

18 Décembre 2024 - 19 Mars 2025

Table des matières

Table des matières 2
Introduction 4
Analyse & Cahier des Charges : Préparation et Collecte des Données 4
Contexte du Projet 4
Objectifs 4
Sources de Données 4
Outils et Méthodologie 4
Analyse et Nettoyage des Données 5
Modélisation des Bases de Données 5
Flux de Données 5
Livrables 5
Accessibilité Numérique 5
Solution & Stack choisis 5
Collecte des datas (ETL) 5
Transformation (ETL) grâce à pyspark et panda 6
Visualisation avec Matplotlib & Seaborn 7
Complétion des données avec Mistral 7
Création de la Base de donnée grâce à PostgreSQL (ETL) 7
Évaluation des options 8
Dockerisation avec Docker 8
Gestion du projet & Gantt 9
Modélisation des données 9
ETL : Récupération des données (Extract) 10
Introduction 10
1. Les images d'empreintes 10
L'idée, afin de tester au mieux la qualité de notre code, a été de récupérer nos données depuis diverses sources : 10
2. Les Métadonnées (Information sur les mammifères) 13
Récupération : 13
ETL : Transformations (Transform) 14
1. Automatisation du remplissage des données animalières 14
2. Problèmes rencontrés et solutions mises en place 14
Encodage des caractères spéciaux mal géré 14
Mauvaise structuration des colonnes dans Excel 14
3. Nettoyage des Données 15
4. Liaison des données 16
5. Visualisation des données 17
Répartition des images par animal 17
Statistiques sur les images 17
Visualisation sous forme de graphiques 17
ETL : Enregistrement des Données (Load) 19
Processus de création de la base de données : 20

Mise en place de Docker Compose:..... 20
Accès à la base de données en utilisant DBeaver..... 21
Création des tables..... 21
Représentation du modèle relationnel :.....22
Intégration des données.....22
Accès aux données..... 22
Conclusion.....23
Perspective :.....23
Sources..... 24
Images :.....24
Métadonnées :.....24
Autres sources..... 24
Glossaire.....25

Introduction

Analyse & Cahier des Charges : Préparation et Collecte des Données

Contexte du Projet

Le projet a été conçu pour répondre à un besoin de l'entreprise WildLens, dont l'objectif est de sensibiliser et d'intéresser les individus à la cause animale. Son objectif : détecter des empreintes via une intelligence artificielle capable de reconnaître des caractéristiques visuelles. Cela pourrait non seulement permettre à chacun de mieux connaître la faune environnante,, mais aussi de suivre les déplacements de certaines espèces. Les données collectées pourraient donc présenter un intérêt scientifique notable pour l'examen de ces espèces.

Objectifs

Cette première phase du projet est singulièrement importante en cela que la qualité de sa réalisation impactera directement la totalité de notre projet. Il s'agira, dans un premier temps, de collecter, nettoyer et structurer un jeu d'images labellisés, puis de modéliser une base de données afin de stocker les informations complémentaires sur les espèces et enfin automatiser le processus ETL (Extract, Transform, Load) pour garantir la qualité et l'accessibilité des données.

Sources de Données

Les données nécessaires pour l'application proviennent de deux principales sources :

- **Photos d'empreintes animales** : Récupérés grâces à diverses sources *iNaturalis*, *nature-tracking*, *OpenAnimalTracks*, *Google Images*
- **Données supplémentaires sur les espèces** : Fournies dans le fichier infos_especes.csv pour chaque espèce ainsi que d'autres sources récupérées sur le net.

Outils et Méthodologie

Pour le bon déroulement de ce projet, nous avons utilisé de nombreux outils tels que Selenium, l'API de Google CustomSearch, Pandas, Grafana, PostgresSQL… afin de nous organiser, nous avons appliqué la méthode Kanban avec Trello, nous avons également effectués un diagramme de Gantt. Nous nous sommes donc inspirés des méthodes agiles acquises lors de nos études.
Nous avons mis un point d'honneur à sécuriser la collecte de données tout en la rendant flexible et adaptée pour une automatisation.

Analyse et Nettoyage des Données

Pour cette étape nous avons vérifié la qualité des données afin d'éliminer les erreurs, ensuite nous avons procédé à une augmentation des données pour enfin finir par structurer ces dernières afin de les rendre exploitables..

Modélisation des Bases de Données

Les informations collectées devront être stockées dans une base de données. Une réflexion sera menée sur la structure de stockage la plus adaptée (SQL, NoSQL), afin de garantir l'accessibilité des données pour l'application et les systèmes tiers.

Flux de Données

- **Importation des données** : Mise en place des flux pour importer les photos d'empreintes et les informations sur les espèces.
- **Automatisation des requêtes** : Préparation des scripts ou requêtes pour la récupération des données dans la base de données en fonction des besoins de l'application.

Livrables

- **Jeu de données d'entraînement, validation et test** : Préparation du jeu de données pour l'entraînement du modèle.
- **Modèle de données** : Création du modèle de données pour stocker les informations sur les empreintes et les espèces.
- **Script de récupération des données** : Création du script permettant d'extraire les informations complémentaires d'une espèce.

Accessibilité Numérique

L'accessibilité des données et des interfaces sera un point clé, avec un focus sur la création de tableaux de bord accessibles à tout public, en prenant en compte les normes d'accessibilité numérique pour les personnes handicapées.

Solution & Stack choisis

Collecte des datas (**E**TL)

Les outils sélectionnés répondent à leurs besoins spécifiques en matière de collecte, de vérification et d'organisation des données pour WildLens

- **Selenium / WebDriver** est un choix préféré pour l'extraction de données des sites Web dynamiques , par exemple, iNaturalist et

NatureTracking, qui reposent lourdement sur JavaScript. Contrairement aux librairies statiques comme BeautifulSoup, Selenium permet de simuler les actions des utilisateurs , par exemple, le défilement infini, les clics sur «Charger plus», ce qui est essentiel pour extraire le contenu préchargé par AJAX.

- **Requests / OpenCV :** idéaux pour les téléchargements légers , par exemple, des images individuelles, avec un contrôle précis des erreurs , tels que le délai d'attente, le code HTTP
- **Pandas :** Permet la manipulation avancée de métadonnées
- (fusion de sources variées, élimination des doublons grâce à drop_duplicates()).
- **CSV :** Format universel pour l'interopérabilité avec les outils tiers
- (Excel, SQL) et la traçabilité des données.
- **Logging :** trace les échecs de téléchargements ressources, hiérarchise les erreurs

Ce stack assure la performance et l'auditabilité tout en respectant les contraintes techniques et légales du projet.

Transformation (E**T**L) grâce à pyspark et panda

L'ETL sera implémenté en utilisant 4 différentes bibliothèques/API Python :

- **PySpark** pour traiter de grandes quantités de données
- **Pandas** pour lire, manipuler et enregistrer des fichiers CSV ainsi que aligner et structurer correctement les données afin d'assurer leur compatibilité avec Excel.
- **Requests** pour effectuer des appels API vers l'intelligence artificielle de Mistral et récupérer dynamiquement des informations sur les espèces animales.
- **Chardet** pour détecter l'encodage du fichier CSV avant de le charger.

Visualisation avec Matplotlib & Seaborn

Dans le cadre de l'analyse de notre dataset, nous avons utilisé **Matplotlib** et **Seaborn** pour créer des visualisations statistiques avancées. Ces bibliothèques Python offrent une flexibilité et une simplicité d'utilisation, permettant de générer des graphiques informatifs et esthétiques.

- Matplotlib : C'est la bibliothèque standard en Python pour la visualisation de données. Elle permet de tracer des histogrammes, des boxplots, des barplots, et bien d'autres types de graphiques personnalisés

Seaborn : Basé sur Matplotlib, Seaborn offre une interface de haut niveau pour dessiner des graphiques statistiques attrayants. Il intègre des statistiques avancées et améliore le rendu visuel des graphiques..

Complétion des données avec Mistral

L'API de **Mistral AI** a été choisie pour :

- Son accès gratuit permettant de générer des descriptions sans coût.
- Sa capacité à répondre en français, en lui imposant un prompt strict.
- Son pré-traitement de texte avancé, permettant de récupérer des données pertinentes sur les animaux

Création de la Base de donnée grâce à PostgreSQL (ET**L**)

Notre projet a besoin d'un système de gestion de stockage des données capable de stocker les résultats finaux (après le modèle d'apprentissage), ainsi que les résultats intermédiaires (Les informations liées au images). Ce système doit se connecter facilement à notre solution ETL et à notre outil de visualisation.
Pour répondre à ces besoins, nous avons testé cinq systèmes de gestion de bases de données souvent utilisés : PostgreSQL, MySQL, MongoDB, SQLite et MariaDB. Notre solution nécessite trois caractéristiques principales de la part d'une base de données :

1. Gestion des requêtes SQL complexes : Pour de la visualisation des données.
2. Capacité à contenir une grande quantité de données : Pour stocker les résultats intermédiaires et finaux.
3. Optimisation pour gérer un grand nombre de requêtes : Pour les transformations, les requêtes de visualisation et l'écriture des données extraites.

Évaluation des options

- MongoDB : MongoDB est une base de données NoSQL, donc l'exécution de requêtes SQL complexes n'est pas possible, ce qui ne correspond pas à nos besoins de visualisation avancée.
- SQLite : SQLite n'est pas optimisé pour traiter d'importants volumes de données ou un grand nombre de requêtes simultanées.
- MariaDB et MySQL : Bien que ces deux peuvent être adaptés, elles offrent des capacités de requêtes moins élevées que celles de PostgreSQL. Leur moteur est également moins optimisé pour gérer un grand nombre de requêtes, ce qui pourrait ralentir notre processus de transformation et de visualisation des données.

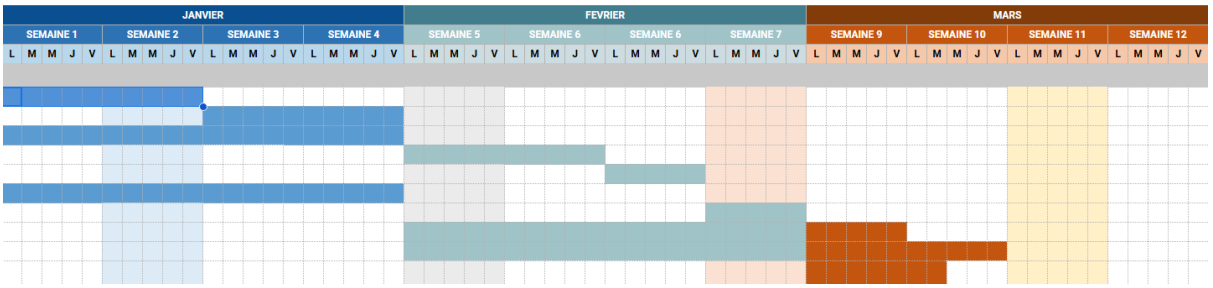
Dockerisation avec Docker

Pour faciliter le déploiement de notre solution, nous avons choisi d'utiliser Docker. La dockerisation permet de créer des conteneurs pour chaque composant de notre stack (Python, PySpark, PostgreSQL, Grafana), assurant ainsi une installation et une mise à jour simplifiées. Cela garantit également la portabilité de notre solution sur différents environnements, que ce soit en développement, en test ou en production (Surtout pour la partie 2 du projet).

Gestion du projet & Gantt

Un diagramme de Gantt a été réalisé pour mieux structurer le projet ainsi que pour suivre l'évolution des tâches. Il a été défini au début du projet mais il a évolué tout au long du projet, au fur et à mesure où nous avons démarré nos différentes tâches, notamment en changeant des affectations.

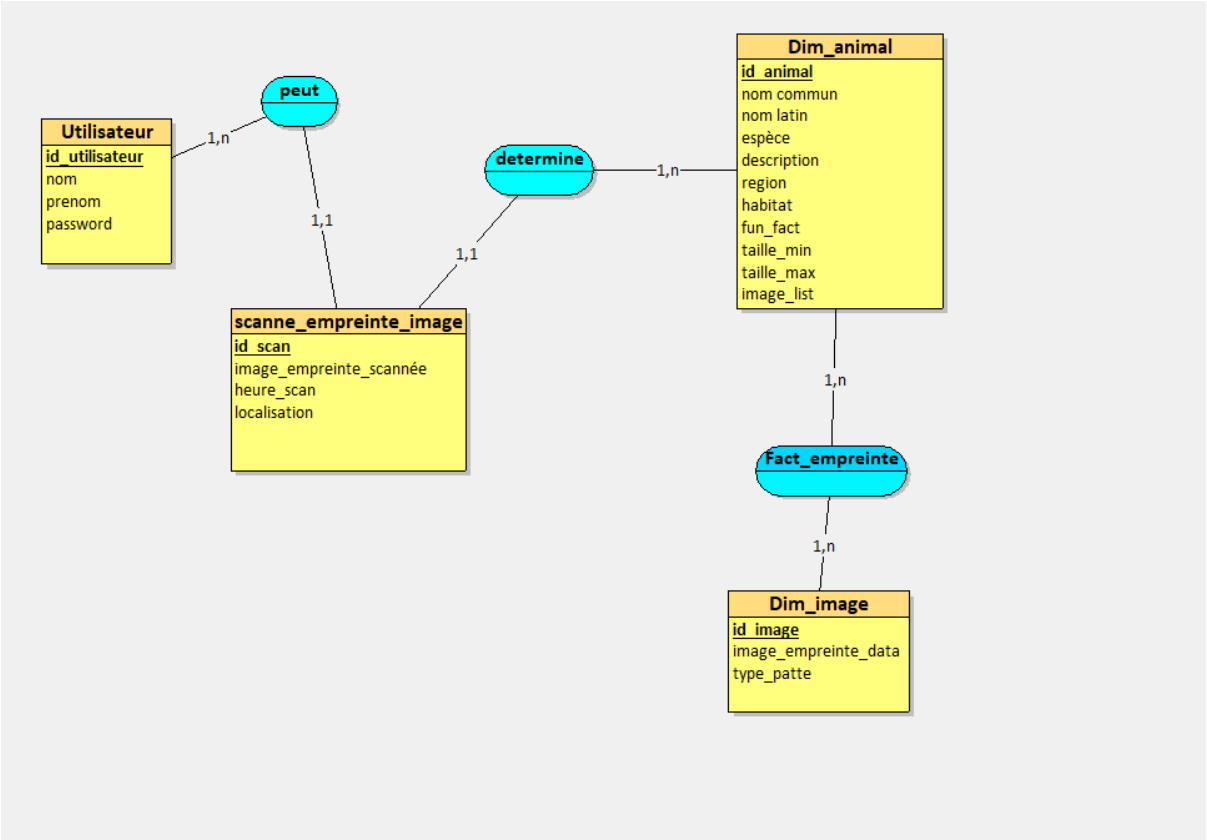
NUMÉRO	TITRE DE LA TÂCHE	PROPRIÉTAIRE DE LA TÂCHE	DATE DE DÉBUT	DATE LIMITE	DURÉE	TÂCHE TERMINÉE (EN %)
1	**Conception et mise en route du projet**					
1.1	Recherche de source pour les info des Mamm	Alban / Carnegie	01/01/25	14/01/25	13	100 %
1.1.1	Cleaning sur les CSV des infos	Alban	14/01/25	01/02/25	17	100 %
1.2	Scraping des images	Loïs	01/01/25	01/02/25	30	100 %
1.3	Fusion des informations avec les animaux	Carnegie	01/02/25	14/02/25	13	80 %
1.3.1	Cleaning du reste + Data vizualisation	Alban / Carnegie	14/02/25	21/02/25	7	70 %
1.4	Mettre en place la BDD (Postgre)	Mehdi	01/01/25	01/02/25	30	70 %
1.4.1	Enregistrement dans la BDD	Mehdi	21/02/25	01/03/25	10	80 %
1.5	Dockerisation	Loïs + Mehdi	01/02/25	08/03/25	37	50 %
1.6	Commencer le rapport + les livrables	Carnegie + Alban	01/02/25	14/03/25	43	100 %
1.7	Data Augmentation (Augmentation des Espèc	Carnegie	01/03/25	10/03/25	9	100 %



Modélisation des données

Au début du projet nous avons pu réaliser un diagramme UML pour nous aider à mieux réfléchir et centraliser les idées tirées du sujet (Ici nous parlons globalement pas seulement sur la partie 1).
Schéma d'architecture logicielle :

Ainsi qu'un modèle pour ensuite créé la base de données.
Modèle conceptuel de donnée (MCD) :



ETL : Récupération des données (Extract)

Introduction

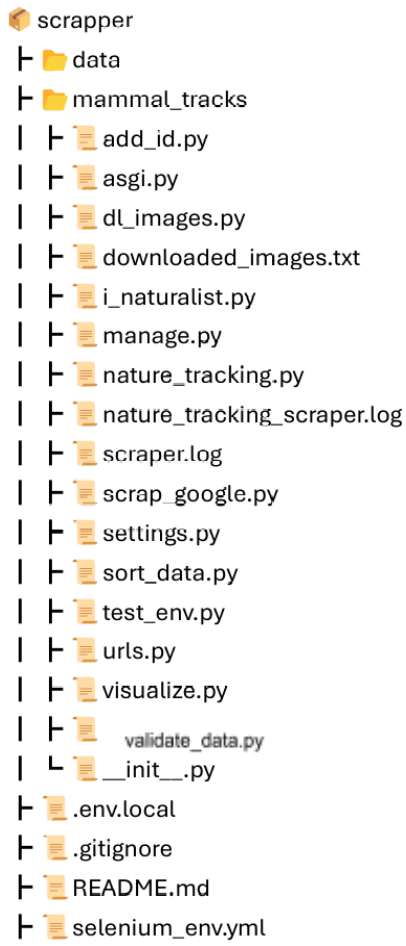
Le processus ETL (Extraction, Transformation, Chargement) est essentiel pour préparer les données relatives aux animaux, à leurs empreintes et à leurs caractéristiques, pour ensuite faire l'entraînement du modèle de reconnaissance d'empreintes animales.

1. Les images d'empreintes

L'idée, afin de tester au mieux la qualité de notre code, a été de récupérer nos données depuis diverses sources :

- **Une source très fiable, OpenAnimalTracks :** Trouvée grâce au site **arxiv.org** et récéptionnée auprès de ses auteurs suite à une demande par mail.
- **Des sources fiables, contenant possiblement des erreurs :** Il s'agit ici des sites *iNaturalist* et *Nature Tracking* dont les données sont fournies et revues par des utilisateurs, il peuvent donc contenir quelques rares erreurs, mais cela reste intéressant.

- **Des sources moins fiables** : Il s'agit de Google Images, l'intérêt était ici triple :
    - Accéder à une plus grande diversité de données
    - Cibler la récupération de données en fonction d'un animal
    - Ajouter du bruit à nettoyer afin de tester au mieux notre nettoyage de données visuel.



→ Cette dernière option impose une vérification préalable des domaines de sites afin de ne récupérer que des images libres de droit depuis des sites dont le fichier robots.txt ne l'interdit pas explicitement. Nous avons implémenté une version rudimentaire de ce tri en fonction des sites mais cette dernière n'est probablement pas suffisante, ainsi les données utilisées dans ce cadre, le sont grâce à l'exception pédagogique en France (Article L122-5 du Code de la propriété intellectuelle) et ne seront pas utilisés pour l'entraînement du modèle, mais seulement lors du test et de l'évaluation de notre code.

Concernant l'organisation du projet, nous avons un fichier pour chaque source, i_naturalist.py pour ***iNaturalist,*** nature_tracking.py pour ***Nature Tracking*** et enfin scrap_google.py pour ***Google Images***.

La collecte de ces données est sécurisée grâce à l'ajout de délais pour respecter les règles des sites, des validations HTTPS, des filtres de domaines…

```
if not any(k in image_url.lower() for k in FOOTPRINT_KEYWORDS) and not any(k in image_snippet for k in FOOTPRINT_KEYWORDS):
    continue

if any(domain in display_link for domain in EXCLUDED_DOMAINS):
    continue
```

Exemple de filtrage dans le fichier scrap_google.py

La fonction check_robots_permission par exemple permet de vérifier les permissions spécifiées dans robot.txt pour les respecter. De plus, une validation des fichiers est effectuée au travers du filtrage des formats jpeg et png, d'une détection des fichiers corrompus via l'ent-tête des images et enfin une taille maximale de 10Mo.

La gestion des erreurs est effectuée au travers d'une relance automatique des requêtes en cas de tentatives échouées (3 tentatives) et un délai de 2.5 secondes entre les requêtes pour éviter le bannissement. Enfin les liens non sécurisés (HTTP) sont filtrés et un proxy configurable est mis en place pour anonymiser les requêtes.

Le fichier scrap_google.py, afin de récupérer des données plus diversifiées permet d'introduire des méthodes qui seront par la suite développées plus en détail par l'ETL.
Nous avons par exemple la vérification des doublons, afin d'éviter de télécharger deux fois une même image avec la fonction load_existing_urls().

De plus, une détection visuelle est effectuée grâce à OpenCv pour détecter les contours d'empreintes.

```
# Détection d'empreintes avec OpenCV
def detect_footprints(image_url):
    try:
        response = requests.get(image_url)
        response.raise_for_status()
        img_array = np.asarray(bytearray(response.content), dtype=np.uint8)
        img = cv2.imdecode(img_array, cv2.IMREAD_COLOR)

        if img is None:
            return False

        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        blurred = cv2.GaussianBlur(gray, (5, 5), 0)
        edges = cv2.Canny(blurred, 50, 150)

        contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        valid_contours = [c for c in contours if cv2.contourArea(c) > 1000]

        return len(valid_contours) > 0

    except requests.exceptions.RequestException as e:
        print(f" ! Erreur lors du téléchargement de l'image : {e}")
        return False
```

Dans cette fonction, on décode l'image avec OpenCV, on la convertit en niveaux de gris (cv2.cvtColor()) et on y ajoute un flou gaussien (cv2.GaussianBlur()) afin d'analyser au mieux l'image. La détection des contours est effectuée avec cv2.Canny() puis un filtre est appliqué pour ne garder que les contours supérieurs à 1000 pixels afin que seul les images les plus susceptibles de contenir des empreintes d'animaux soient gardés.

Les fichiers relatifs à iNaturalist et Nature Tracking sont eux bien différents. Leurs scripts respectifs se concentrent essentiellement sur l'automatisation d'action sur les pages afin de charger toutes les données avant de d'extraire l'url des images ciblées ainsi que le nom de l'animal associé.
La plupart de ces scripts sont relativement modulaires et peuvent aisément être appelés en local via un fichier externe ou en consommant une API, enfin, le fichier

sort_data.py nous a permis de trier et normaliser les données afin de pouvoir les transmettre directement à notre base de données.

Au total, nous avons récupéré ***940 images labellisées prêtes à être utilisées pour l'entraînement de notre modèle*** grâce à nos scrappers. Cela additionné aux 2 335 images récupérées avec le dataset Open Animal Tracks, fait un total de ***3 275 images à envoyer à l'ETL.***

2. Les Métadonnées (Information sur les mammifères)

Récupération :

L'objectif ici est de garnir du maximum possible le fichier CSV des noms des animaux mammifères pour accroître notre base de données d'animaux identifiables. Pour cela plusieurs sources fiables ont été utilisé :

Infos_especes : Liste d'espèces d'animaux fournir par l'école et qui sert de guide aux informations des espèces à recueillir

Animal information dataset (kaggle) : Cet ensemble de données englobe un large éventail d'attributs relatifs à diverses espèces animales du monde entier. L'ensemble de données comprend principalement des champs tels qu'Animal, Taille (cm), Poids (kg), Couleur, Durée de vie (années), Régime alimentaire, Habitat, Prédateurs, Vitesse moyenne (km/h), Pays trouvés, Statut de conservation, Famille, Période de gestation (jours), Vitesse de pointe (km/h), Structure sociale et Progéniture par naissance.

Diconimoz : Le site possède une liste des mammifères sauvages classés de A à Z, couvrant diverses espèces terrestres.

Instinct Animal : Le site propose une liste des mammifères classés par ordre alphabétique de A à Z. Chaque animal comporte une fiche descriptive complète et détaillée avec des photos et des illustrations.

L'ensemble des informations de ces sources ont été récupérées puis stockées dans un même fichier csv.
A présent nous disposons d'une grande liste de noms d'espèces d'animaux mammifères mais nous ne disposons pas de la totalité des informations à savoir la description, le nom latin, la famille, les tailles minimales et maximales, la région, l'habitat et le fun fact de ces animaux.

ETL : Transformations (Transform)

Cette partie va principalement traiter la partie de transformation des données précédemment récupérés grâce au scraping.

1. Automatisation du remplissage des données animalières

L'objectif est de faciliter et automatiser le remplissage des informations manquantes du fichier csv des différentes espèces animales à partir du nom de l'espèce déjà fournie. En utilisant l'intelligence artificielle (IA) Mistral AI, les informations manquantes regroupées par colonnes sont complétées de manière fiable, tout en assurant un bon formatage pour une exploitation facile du csv.
Pendant le développement, plusieurs problèmes techniques se sont présentés, et tous ont été résolus au fur et a mesuré résolu.

2. Problèmes rencontrés et solutions mises en place

Encodage des caractères spéciaux mal géré

Problèmes identifiés : Les accents et caractères spéciaux (ex : é, à, ç) étaient mal affichés sous Excel, générant des symboles incompréhensibles.
Des erreurs de lecture survenaient (« UnicodeDecodeError »), ce qui bloquait l'ouverture du fichier.

Solutions mises en place :

- Utilisation de l'outil **Chardet** pour pouvoir identifier l'encodage du fichier source.
- La lecture du fichier avec l'encodage adapté (ISO-8859-1 ou UTF-8).
- Enregistrement final en UTF-8-SIG, qui assure une compatibilité optimale avec Excel.

```
# Détecter l'encodage du fichier CSV
with open("animaux2.csv", "rb") as f:
    result = chardet.detect(f.read(100000))
    detected_encoding = result["encoding"]
    print(f"🔍 Encodage détecté : {detected_encoding}")
```

Mauvaise structuration des colonnes dans Excel

Problèmes identifiés :

Toutes les données étaient fusionnées dans une seule colonne.
Le séparateur de colonnes (« , » ou « ; ») était mal interprété.
Solutions mises en place :

- Détection automatique du séparateur correct (; ou ,).
- Sauvegarde du fichier CSV en forçant le bon délimiteur (;), pour garantir un affichage fluide dans Excel.

```
# Charger le fichier CSV avec le bon séparateur
try:
    df = pd.read_csv("animaux2.csv", encoding=detected_encoding, sep=";", quotechar='"', on_bad_lines="skip")
    if df.shape[1] == 1:  # Si une seule colonne est détectée, essayer avec ','
        df = pd.read_csv("animaux2.csv", encoding=detected_encoding, sep=",", quotechar='"', on_bad_lines="skip")
        print("⚠️ Le séparateur correct semble être ','.")
except Exception as e:
    print(f"❌ Erreur lors du chargement : {e}")
    exit()
```

3. Nettoyage des Données

Le nettoyage des données est important pour garantir leur qualité avant la fusion, dans cette partie on va se pencher sur le nettoyage du précédent CSV.Dans cette étape nous utiliserons **Panda** pour lire les donnés et **PySpark** pour les manipulations de données

Filtrage par famille :

- Le Data Frame est ensuite filtré pour ne conserver que les animaux appartenant aux familles de mammifères terrestres, spécifiées dans la liste mammiferes_terrestres.
- Le nombre total d'animaux après ce filtrage est affiché.

Suppression des doublons :

- Utilisation de fonctions comme dropDuplicates() pour éliminer les doublons dans les données, pour qu'il n'y est qu'une source d'information par espèces.

Exemple:

```
Les doublons (basés sur la colonne 'Animal') :
+--------------+-----+
|Animal        |count|
+--------------+-----+
|Wombat        |2    |
|Kangourou roux|2    |
|Yak           |2    |
|Serval        |2    |
|Dingo         |2    |
|Renard roux   |2    |
|Oryx d'Arabie |2    |
|Okapi         |2    |
+--------------+-----+
```

Traitement des valeurs manquantes :

- Suppression de valeurs par défaut ou élimination des lignes manquantes avec na.drop() pour éviter les problèmes liés aux données incomplètes.

Nettoyage des Colonnes :

- Utilisation de fonctions comme regexp_replace() et trim() pour supprimer les espaces inutiles et les caractères spéciaux, assurant ainsi une cohérence dans les valeurs.

Normalisation des Valeurs :

- Conversion des données numériques (comme la taille des animaux) dans une unité standard (ici on va tout transformer en centimètre).
- Normalisation des noms des animaux et des espèces pour éviter les incohérences (par exemple, tout mettre en minuscules avec str.lower()).

4. Liaison des données

Les Informations propres des animaux sont enregistrées dans nôtre cas en “Apache parquet”. Ce type de fichier est plus optimisé et ses performances sont meilleures par rapport au CSV . On va ensuite liés ces informations au empreintes:

- Les données provenant des différentes sources (photos des empreintes et métadonnées des animaux) sont liées pour créer des ensembles de données complets.
- Chaque entrée dans la table d’information est liée à une liste d'images d’empreinte grâce à la colonne Animal..

5. Visualisation des données

L'objectif de la visualisation des données est de mieux appréhender les métadonnées liées aux images collectées pour essayer d'optimiser l'entraînement du modèle de reconnaissance d'empreintes animales. Grâce aux analyses nous pouvons identifier les espèces avec peu d'images et s'assurer qu'il y a une ressemblance entre tous les éléments du dataset avant l'apprentissage automatique.

Répartition des images par animal

Nous avons analysé le nombre d'images disponibles par espèce afin de déterminer si certaines catégories sont sous-représentées. Voici les principales observations :

- Certaines espèces possèdent un grand nombre d'images, ce qui est idéal pour l'apprentissage du modèle.
- D'autres espèces ne disposent que de très peu d'images, ce qui peut poser des problèmes lors de l'entraînement du modèle.

Statistiques sur les images

Voici quelques données extraites de notre analyse :

- Moyenne d'images par animal : 19.6 images.
- Nombre d'animaux avec moins de 10 images : 3 animaux.

Un nombre minimal d'images par animal sera mis en place pour assurer un entraînement optimal du modèle.

```
### Statistiques sur les images

- **Moyenne d'images par animal** : 19.60 images.
- **Nombre d'animaux avec moins de 10 images** : 3 animaux.

Un nombre minimal d'images par animal sera mis en place pour assurer un entraînement optimal du modèle.
```

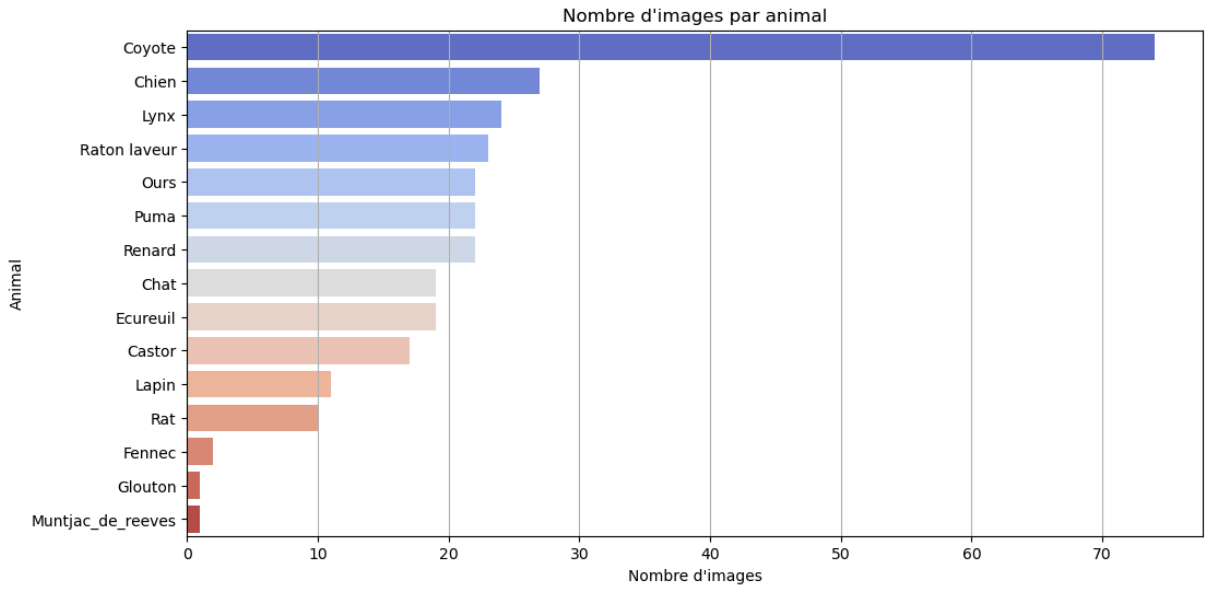
Screen réalisé sur des données de test (Non finale)

Visualisation sous forme de graphiques

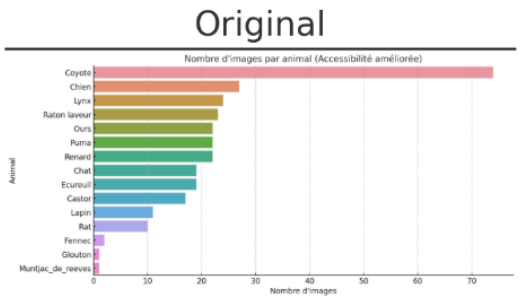
Afin de rendre ces observations plus lisibles, nous avons réalisé plusieurs graphiques :

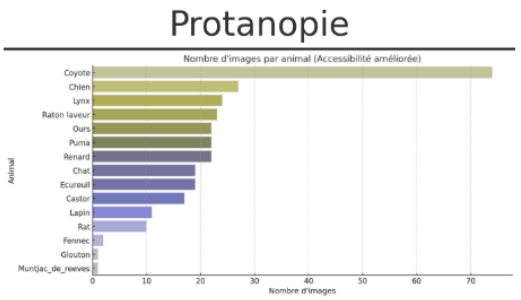
- Histogramme de la répartition des images par espèce : permet d'observer la densité des images disponibles pour chaque animal.
- Diagramme du nombre d'animaux avec peu d'images : montre la proportion d'espèces ayant moins de 10 images.

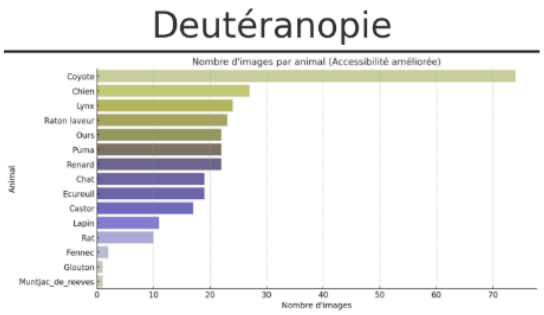
Ces analyses sont importantes pour garantir une base de données optimisée avant l'entraînement du modèle IA de reconnaissance d'empreintes animales.

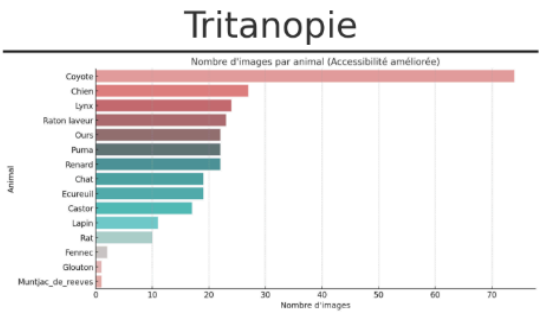


Screen réalisé sur des données de test (Non finale)









Voici différentes simulations de daltonisme appliquées au graphique à partir d'un script pour générer les couleurs les mieux adaptés aux différentes types de daltonismes :

1. **Protanopie (insensibilité au rouge)** : Les couleurs rouges deviennent jaunâtres et sont difficiles à différencier des verts et bruns. Cela peut poser problème pour distinguer certaines barres.

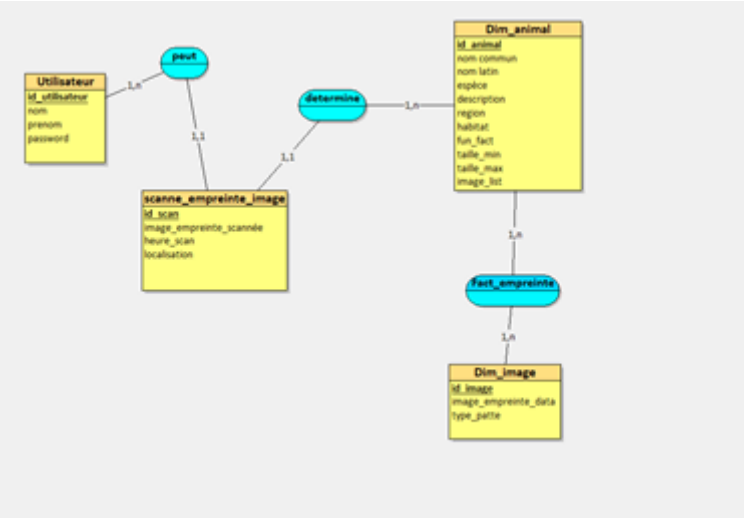
2. **Deutéranopie (insensibilité au vert)** : Les couleurs vertes et rouges sont perçues de manière similaire, ce qui entraîne une perte de contraste entre certaines barres du graphique.

3. **Tritanopie (insensibilité au bleu)** : Les bleus et jaunes sont difficiles à différencier. Les couleurs semblent plus atténuées mais restent relativement distinctes.

ETL : Enregistrement des Données (Load)

Nous avons établi une base de données relationnelle conçue pour rassembler et structurer diverses informations, y compris celles concernant les utilisateurs, les empreintes digitales et les animaux.
Avant de nous plonger dans l'aspect technique, nous avons d'abord défini un MCD afin d'établir solidement la structure relationnelle.
Analyse du MCD :

- **Utilisateur** : Cette table renferme les données cruciales des utilisateurs , telles que l'identifiant, le nom, le prénom et le mot de passe.
- **Scanne_empreinte_image** : Ce lieu contient les empreintes numérisées, accompagnées d'informations sur l'image de l'empreinte, le moment du scan, la position géographique et l'utilisateur lié.
- **Dim_animal** : Ce segment contient toutes les informations relatives aux animaux, couvrant leur dénomination commune et scientifique, ainsi que des précisions plus pointues telles que leur milieu de vie , leur zone géographique et même des faits divertissants
- **Dim_image** : Ce tableau énumère les diverses traces d'animaux, accompagnées de leurs informations visuelles et du genre de patte
- **Relations** :
  Un utilisateur peut effectuer plusieurs analyses.
  Chaque analyse est associée à une empreinte d'animal précise.
  Les traces animales sont liées aux données concernant les animaux grâce à la table fact_empreinte.



Processus de création de la base de données :

Mise en place de Docker Compose:

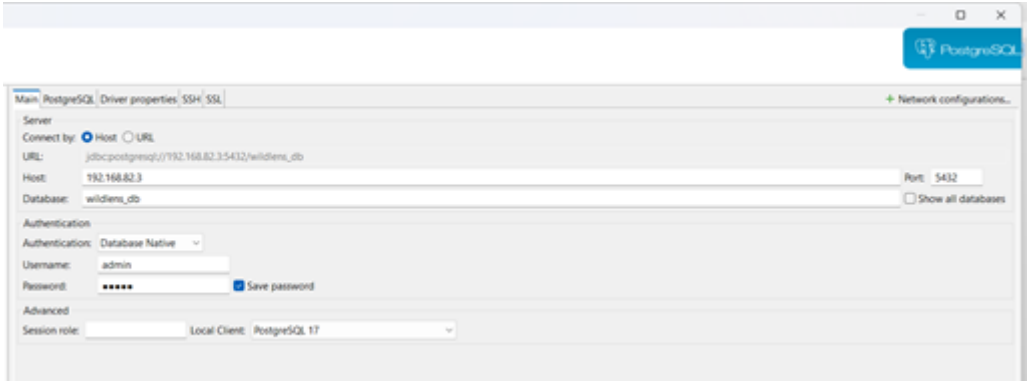
Nous avons mis en place la base de données PostgreSQL en utilisant Docker grâce à un fichier [docker-compose.yaml]() .Ce document renferme tous les réglages essentiels pour paramétrer le service PostgreSQL, établir un volume de données durable et modifier les configurations réseau.Cette configuration garantir simultanément la protection des données et leur accessibilité.

```
GNU nano 7.2
version: '3.8'
services:
  wildlens_db:
    image: postgres
    container_name: wildlens_database
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin
      POSTGRES_DB: wildlens_db
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    restart: always

volumes:
  pgdata:
```

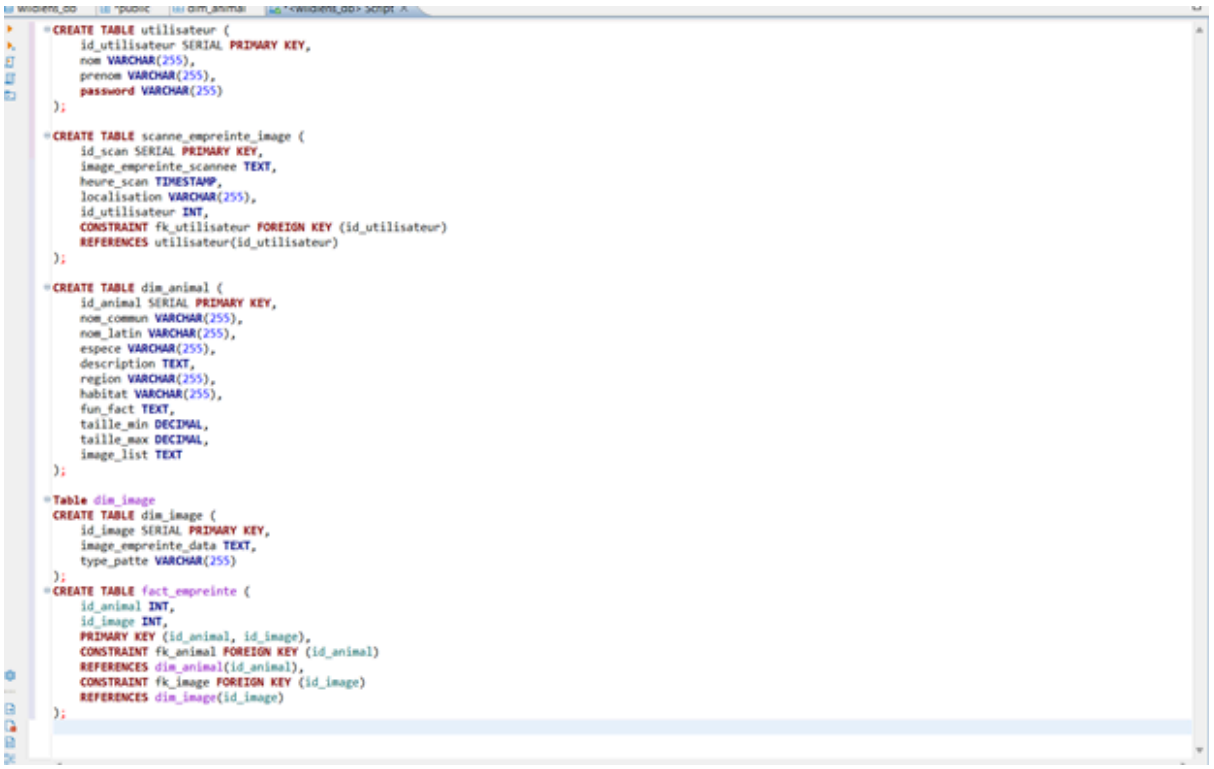
Accès à la base de données en utilisant DBeaver

Pour simplifier la gestion de notre base de données, nous avons opté pour DBeaver. L'établissement de la connexion a la base de données PostgreSQL s'est effectué en suivant des étapes simples , en fournissant les informations de connexion telles que l'hôte, le port, le nom de la base de données l'utilisateur et le mot de passe.



Création des tables

La création des différentes tables a été effectuée grâce à une suite de requêtes SQL exécutées directement via DBeaver.



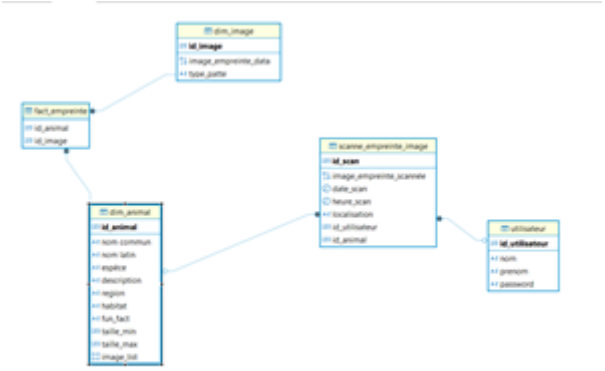
```
CREATE TABLE utilisateur (
    id_utilisateur SERIAL PRIMARY KEY,
    nom VARCHAR(255),
    prenom VARCHAR(255),
    password VARCHAR(255)
);

CREATE TABLE scanne_empreinte_image (
    id_scan SERIAL PRIMARY KEY,
    image_empreinte_scannee TEXT,
    heure_scan TIMESTAMP,
    localisation VARCHAR(255),
    id_utilisateur INT,
    CONSTRAINT fk_utilisateur FOREIGN KEY (id_utilisateur)
    REFERENCES utilisateur(id_utilisateur)
);

CREATE TABLE dim_animal (
    id_animal SERIAL PRIMARY KEY,
    nom_commun VARCHAR(255),
    nom_latin VARCHAR(255),
    espece VARCHAR(255),
    description TEXT,
    region VARCHAR(255),
    habitat VARCHAR(255),
    fun_fact TEXT,
    taille_min DECIMAL,
    taille_max DECIMAL,
    image_list TEXT
);

Table dim_image
CREATE TABLE dim_image (
    id_image SERIAL PRIMARY KEY,
    image_empreinte_data TEXT,
    type_patte VARCHAR(255)
);
CREATE TABLE fact_empreinte (
    id_animal INT,
    id_image INT,
    PRIMARY KEY (id_animal, id_image),
    CONSTRAINT fk_animal FOREIGN KEY (id_animal)
    REFERENCES dim_animal(id_animal),
    CONSTRAINT fk_image FOREIGN KEY (id_image)
    REFERENCES dim_image(id_image)
);
```

Représentation du modèle relationnel :



Intégration des données

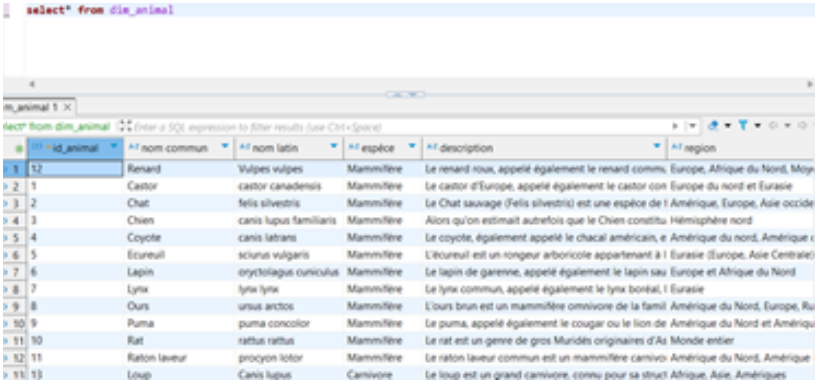
Pour alimenter la base de données avec des données sur les animaux, nous avons utilisé des instructions "INSERT INTO".
Ces requêtes facilitent l'insertion d'informations précises, telles que les noms des animaux, leurs caractéristiques, leurs habitats et même les photos associées à leurs empreintes.

```
INSERT INTO Dim_animal (id_animal, "nom commun", "nom latin", espèce, description, region, habitat, fun_fact, taille_min, taille_max, image_list)
VALUES
(9, 'Puma', 'puma concolor', 'Mammifère',
 'Le puma, appelé également le cougar ou le lion de montagne, est un grand mammifère carnivore de la famille des félins.',
 'Amérique du Nord et Amérique du Sud',
 'Le puma vit dans les forêts, les prairies, les zones marécageuses.',
 'Le puma est un animal qui peut s''apprivoiser facilement. Certains fermiers en Argentine apprivoisent les félins et les laissent vivre en semi-liberté à la manière d
 100, 230,
 '{Mammifères\\Puma\\Cougar-Tracks-10.jpg, Mammifères\\Puma\\Cougar-Tracks-9.jpg, Mammifères\\Puma\\Cougar-Tracks.jpg, Mammifères\\Puma\\Mountain-Lion-Tracks-(1).jpg, M
```

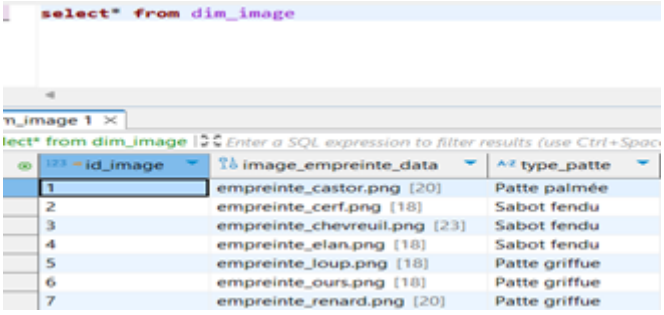
```
INSERT INTO Dim_animal (id_animal, "nom commun", "nom latin", espèce, description, region, habitat, fun_fact, taille_min, taille_max, image_list)
VALUES
(2, 'Chat', 'felis silvestris', 'Mammifère',
 'Le Chat sauvage (Felis silvestris) est une espèce de félins du genre Felis présent dans divers types d''habitats. Son aire de répartition couvre l''Europe, l''Asie o
 'Amérique, Europe, Asie occidentale et Afrique',
 'Partout',
 'Un chat ne peut pas voir directement sous son nez !',
 46, 51,
 '{Mammifères\\Chat\\Housecat-Tracks-(1).jpg, Mammifères\\Chat\\Housecat-Tracks-2 (1).jpg, Mammifères\\Chat\\Housecat-Tracks-2.jpg, Mammifères\\Chat\\large (1).jpg, Ma
```

Accès aux données

Nous avons effectué des requêtes SELECT* sur les tables dim_animal et dim_image. Ce qui nous a permis de valider que les informations étaient bien enregistrées et qu'elles apparaissent comme prévu dans DBeaver.



id_animal	nom commun	nom latin	espèce	description	region
12	Renard	Vulpes vulpes	Mammifère	Le renard roux, appelé également le renard commu	Europe, Afrique du Nord, Moy
1	Castor	castor canadensis	Mammifère	Le castor d'Europe, appelé également le castor con	Europe du nord et Eurasie
2	Chat	felis silvestris	Mammifère	Le Chat sauvage (Felis silvestris) est une espèce de l	Amérique, Europe, Asie occide
3	Chien	canis lupus familiaris	Mammifère	Alors qu'on estimait autrefois que le Chien constitu	Hémisphère nord
4	Coyote	canis latrans	Mammifère	Le coyote, également appelé le chacal américain, e	Amérique du nord, Amérique
5	Ecureuil	sciurus vulgaris	Mammifère	L'écureuil est un rongeur arboricole appartenant à l	Eurasie (Europe, Asie Centrale)
6	Lapin	oryctolagus cuniculus	Mammifère	Le lapin de garenne, appelé également le lapin sau	Europe et Afrique du Nord
7	Lynx	lynx lynx	Mammifère	Le lynx commun, appelé également le lynx boréal, l	Eurasie
8	Ours	ursus arctos	Mammifère	L'ours brun est un mammifère omnivore de la famil	Amérique du Nord, Europe, Ru
9	Puma	puma concolor	Mammifère	Le puma, appelé également le cougar ou le lion de	Amérique du Nord et Amériqu
10	Rat	rattus rattus	Mammifère	Le rat est un genre de gros Muridés originaires d'As	Monde entier
11	Raton laveur	procyon lotor	Mammifère	Le raton laveur commun est un mammifère carnivo	Amérique du Nord, Amérique
13	Loup	Canis lupus	Carnivore	Le loup est un grand carnivore, connu pour sa struct	Afrique, Asie, Amériques



```
select* from dim_image
```

id_image	image_empreinte_data	type_patte
1	empreinte_castor.png [20]	Patte palmée
2	empreinte_cerf.png [18]	Sabot fendu
3	empreinte_chevreuil.png [23]	Sabot fendu
4	empreinte_elan.png [18]	Sabot fendu
5	empreinte_loup.png [18]	Patte griffue
6	empreinte_ours.png [18]	Patte griffue
7	empreinte_renard.png [20]	Patte griffue

Conclusion

Grâce à l'utilisation de Docker pour le déploiement de la base de données sur DBeaver, nous avons pu établir une base de données structurée. L'intégration des données dans la table dim_animal a permis la mise en place de la base de données.

Perspective :

Axes d'amélioration :

- Grafana : Utiliser un outil de visualisation plus avancé et directement après l'insertion en BDD.
- Scrapping des CSV : Automatisation complète de la récupération des métadonnées.
- Mise en place de CI-CD pour simplifier et accélérer le déploiement. (Passer de GitHub à GitLab)
- Dockeriser l'ensemble pour gagner en efficacité et éviter les erreurs.

Sources

Images :

https://www.inaturalist.org/
https://naturetracking.com/

Métadonnées :

https://www.kaggle.com/datasets/iamsouravbanerjee/animal-information-dataset?resource=download
https://www.diconimoz.com/animaux-fiches/mammiferes/liste/
https://www.instinct-animal.fr/liste-mammiferes/

Autres sources

Code de la propriété intellectuelle :
https://www.legifrance.gouv.fr/codes/article_lc/LEGIARTI000048603495
Débogage / aide au développement :
https://copilot.microsoft.com/
Modification et création d'un CSV avec erreur :
https://chatgpt.com/
AI utilisés pour la complétion du CSV :
https://mistral.ai/fr

Glossaire

ETL - Extract Transform Load (Des données)

BDD - Base de donnés

CSV - Format de stockage de données

Parquet - Format de fichier de données open-source

API - Selon la CNIL, “Une API (application programming interface ou « interface de programmation d’application ») est une interface logicielle qui permet de « connecter » un logiciel ou un service à un autre logiciel ou service afin d’échanger des données et des fonctionnalités.”